

HelioX RAG 2 . 1 — Technical Architecture Blueprint

1 . System Overview

HelioX RAG 2 . 1 is a Retrieval-Guided Multi-Agent Deliberative Architecture designed for: - Low-latency selective retrieval - Reduced token consumption - High-grounded reasoning - Structured hallucination mitigation

Core Principles: 1. Deterministic preprocessing before probabilistic reasoning. 2 Metadata-first retrieval, embedding-second retrieval. 3. Selective expert activation instead of full-context reasoning. 4 . Evidence-bound generation (no uncited claims). 5 . Confidence-aware output.

2 . Macro Architecture

User Query ↓ Query Analyzer (Intent + Constraint Extractor) ↓ Routing Engine (Expert Selector)
Worker Agents (Chunk Reasoners) ↓ Adjudication Engine (Conflict + Authority Resolver) ↓ Answer
Composer (Citation-Bound Output) ↓ Confidence Scorer + Session Memory

3 . Phase 1 — Deterministic Sanitization Layer

3 . 1 Document Ingestion Pipeline

Input Types: - PDF (native + scanned) - CSV - TXT - HTML - OCR documents

Pipeline Steps: 1. Format Detection 2. Structural Parsing - Section detection - Table detection - Footnote mapping 3. Noise Removal - Header/footer stripping - Page number removal - Encoding normalization 4. Parent-Child Graph Construction - Section → Subsection → Paragraph → Sentence
Table → Row → Cell mapping

Output: Structured Document Graph (JSON)

3 . 2 Ontology & Terminology Standardization

Instead of rewriting content via LLM: - Domain dictionary mapping - Synonym expansion index
Abbreviation resolution table

Output: Normalized semantic tokens without altering original meaning.

4 . Phase 2 — Multi-Vector Expert Profiling Layer

Each `8 0 0 - 1 2 0 0` token chunk produces:

- 1 . Primary Semantic Embedding
- 2 . Entity Vector (NER extracted entities)
- 3 . Hypothetical Question Embeddings (HyDE generation)
- 4 . Constraint Metadata:
- 5 . Temporal
- 6 . Geographic
- 7 . Regulatory
- 8 . Applicability scope

Storage Architecture: - Vector DB (FAISS / Qdrant / Weaviate) - Metadata DB (PostgreSQL / Elasticsearch) - Cache (Redis)

Expert Profile Schema: { `chunk_id`, `semantic_embedding`, `entity_list`, `hyde_questions`, `constraint_tags`, `authority_score`, `timestamp`, `parent_section` }

5 . Phase 3 — Intent-Driven Routing Engine

5 . 1 Query Decomposition

Input Query → Structured Query Object

{ `fact_components`, `reasoning_components`, `constraint_filters`, `entity_targets` }

Steps: 1 . Intent classification (fact lookup / comparison / synthesis / compliance check) 2 . Entity extraction 3 . Temporal & jurisdiction filter extraction

5 . 2 Hybrid Retrieval Strategy

Stage 1: Metadata Filtering Stage 2: Entity Matching Stage 3: Vector Similarity Search Stage 4: Fallback Broad Retrieval (if recall < threshold)

Top-K Selection: Dynamic K based on confidence score.

6 . Phase 4 — Parallel Worker Agents

Each worker receives: - One chunk - Structured query - Mandatory citation format

Worker Output Schema: { claim, supporting_text_span, reasoning_trace, confidence_local }

Rules: - No claim without span index - No external knowledge allowed

7 . Phase 5 — Adjudication Engine

7 . 1 Conflict Detection

- Semantic contradiction detection
- Overlap inconsistency scoring

If conflict detected: → Trigger re-evaluation with stricter extraction prompt

7 . 2 Authority Weighting

Weighted score = $(w_1 \times \text{recency}) + (w_2 \times \text{source_rank}) + (w_3 \times \text{citation_density}) + (w_4 \times \text{worker_confidence})$

Highest weighted consensus wins.

8 . Phase 6 — Evidence-Bound Answer Composer

Rules: 1. Every factual sentence must map to citation. 2. If coverage < threshold → output "Insufficient evidence." 3. Structured output template enforced.

Output Format: - Direct Answer - Supporting Evidence (citations) - Constraints Applied - Confidence Score

9 . Phase 7 — Confidence & Self-Validation Layer

Metrics: 1 . Retrieval coverage score 2. Citation density 3 Conflict severity score 4 Cross-worker agreement score

Global Confidence = weighted aggregation.

If confidence < X: → Trigger expanded retrieval OR request clarification from user.

1 0 . Session Memory (Controlled)

Memory Stores Only: - Validated structured summary - Confirmed constraints - User preference context

Memory Never Stores: - Raw LLM speculation - Unverified claims

1 1 . Hallucination Mitigation Strategy

- 1 . Deterministic preprocessing
- 2 . Retrieval-first reasoning
- 3 . Mandatory citation enforcement
- 4 . Cross-agent contradiction detection
- 5 . Confidence threshold gating
- 6 . Fallback retrieval safety net

Expected Outcome: Hallucination significantly reduced but not mathematically zero.

1 2 . Scalability Considerations

- Async parallel worker execution
 - Embedding caching
 - Batch ingestion pipeline
 - Horizontal scaling of worker layer
 - Vector sharding for large corpora
-

1 3 . Evaluation Framework

Offline Metrics: - Retrieval Recall@K - MRR - Citation Accuracy - Faithfulness Score

Online Metrics: - Latency per query - Token consumption - Conflict trigger frequency - Confidence calibration curve

1 4 . Architectural Improvements Over 2 . 0

- 1 . Hybrid deterministic + semantic routing
 - 2 . Fallback recall safety layer
 - 3 . Confidence-aware gating
 - 4 . Authority-weighted adjudication
 - 5 . Strict evidence-bound generation
 - 6 . Structured worker schema
-

1 5 . End-to-End Workflow Diagrams

Below are clear execution flows for: 1 . Data Upload (Ingestion Path) 2 . User Query (Inference Path)

A. DATA UPLOAD WORKFLOW (PDF / CSV / TXT / OCR)

[User Uploads File] ↓ [File Type Detection] ↓ [Parser Layer] - PDF parser / OCR engine - HTML reader - HTML cleaner ↓ [Noise Removal Engine] - Header/footer stripping - Page number removal - Encoding normalization ↓ [Structural Decomposition] - Section detection - Table extraction - Parent-child graph creation ↓ [Semantic Standardization] - Abbreviation resolution - Ontology mapping - Synonym expansion ↓ [Chunking Engine] - 800-1200 token adaptive chunks - Maintain parent section reference ↓ [Expert Profiling Generator] For each chunk: • Semantic embedding • Entity extraction • HyDE question generation • Constraint tagging (time/geo/scope) ↓ [Storage Layer] - Vector DB (semantic HyDE vectors) - Metadata DB (entities + constraints) - Redis (hot cache index) ↓ [Blackboard Updated] ↓ System Ready for Query

B. USER QUERY WORKFLOW (INFERENCE PATH)

[User Submits Query] ↓ [Query Analyzer] - Intent classification - Entity extraction - Constraint extraction - Fact vs Reasoning separation ↓ [Structured Query Object Created] ↓ [Routing Engine] - Hybrid routing
Stage 1: Metadata filtering (constraints) Stage 2: Entity matching Stage 3: Vector similarity search
Stage 4: Confidence check

If recall < threshold: → Trigger fallback broad retrieval ↓ [Top-K Expert Chunks Selected] ↓ [Parallel Agents Activated] Each worker receives: • One chunk • Structured query • Citation rules

Worker Output: • Claim • Supporting span • Local confidence ↓ [Adjudication Engine] - Detect contradictions - Authority weighting - Agreement scoring

If conflict detected: → Re-evaluation prompt ↓ [Answer Composer] - Evidence-bound generation
Structured output format - Inline citations ↓ [Confidence Scoring Layer] - Retrieval coverage - density - Worker agreement score

If confidence < X: → Expand retrieval OR ask user clarification ↓ [Final Answer Delivered] ↓ [Memory Update] - Store validated structured summary only

C. SYSTEM CONTROL LOGIC (SIMPLIFIED FLOW)

Upload → Sanitize → Profile → Index → Ready Query → Analyze → Route → Parallel Reason
Validate → Respond

Final Statement

HelioX RAG 2.1 operates as two major pipelines: 1. A deterministic ingestion pipeline that builds structured expertise. 2. A selective multi-agent inference pipeline that activates only relevant knowledge.

This separation is what enables speed, cost efficiency, and strong hallucination mitigation.